

TP1 - Aplicación de compras online

Materia: Aplicaciones Móviles

Profesor: Medina, Sergio Daniel

Alumno: Gauna, Luciano Ezequiel

Comisión: ACN4BV

1. Descripción general del proyecto

Para este primer trabajo se plantea el diseño inicial de una aplicación móvil orientada a la compra de productos de supermercado. Como propuesta general del producto, se diseñaron cinco pantallas principales que representan distintas etapas del flujo de uso de la aplicación:

- Login
- Explorar
- Categorías
- Detalle de producto
- Mi carrito

Estas pantallas fueron planteadas mediante wireframes y diseños visuales, y se incluyen como parte del proyecto para mostrar una idea general del producto final. La cantidad de pantallas y funcionalidades podría modificarse en el futuro según el avance de la cursada, las correcciones recibidas y las necesidades del desarrollo.

Para esta primera implementación se decidió desarrollar únicamente la pantalla **Mi carrito**, ya que permite trabajar con una buena variedad de componentes visuales e interacciones dentro de una sola pantalla.

2. Pantalla seleccionada: Mi carrito

La pantalla seleccionada para desarrollar fue **Mi carrito**. Esta pantalla representa el momento en el que el usuario revisa los productos que agregó antes de avanzar al pago.

Se eligió esta pantalla porque permite incorporar varias interacciones relacionadas con el objetivo principal de la aplicación, como modificar cantidades, eliminar productos, calcular un total estimado y mostrar estados según el contenido del carrito.

Además, es una pantalla adecuada para cumplir con los requisitos técnicos del trabajo, ya que permite utilizar ConstraintLayout, LinearLayout, TextView, Button, imágenes de productos, eventos de usuario y elementos creados dinámicamente desde Java.

3. Objetivo de la pantalla

El objetivo de la pantalla **Mi carrito** es permitir que el usuario pueda:

- Ver los productos agregados al carrito.
- Consultar el nombre, presentación, imagen y precio de cada producto.
- Aumentar o disminuir la cantidad de cada producto.
- Eliminar productos del carrito.
- Consultar el total estimado del pedido.
- Visualizar un mensaje cuando el carrito queda vacío.

La pantalla funciona como un paso previo al pago, donde el usuario puede revisar y ajustar su compra antes de continuar.

4. Estructura visual implementada

La pantalla fue construida utilizando XML. Se usó un `ConstraintLayout` como contenedor principal, ya que permite ubicar los elementos principales de la pantalla mediante restricciones.

Dentro de la pantalla se incluyeron:

- Un título principal: **Mi carrito**.
- Una línea divisoria debajo del encabezado.
- Un `ScrollView` para contener la lista de productos.
- Un `LinearLayout` vertical para organizar los productos del carrito.
- Diferentes `LinearLayout` horizontales para estructurar cada producto.
- Un botón principal **Ir al pago**.
- Una barra de navegación inferior con íconos vectoriales.

Cada producto del carrito está compuesto por una imagen, información textual, controles de cantidad, precio y botón para eliminar el producto.

5. Uso de layouts

Para la estructura general de la pantalla se utilizó un `ConstraintLayout`, que permite fijar elementos como el título, el botón de pago y la barra inferior en posiciones específicas.

También se utilizaron `LinearLayout` con distintas orientaciones:

LinearLayout vertical:

Se usó para apilar los productos dentro del carrito y también para ordenar elementos internos, como nombre, unidad y controles de cantidad.

LinearLayout horizontal:

Se usó para armar cada fila de producto, colocando la imagen, la información y el precio/eliminar en una misma línea.

Esta combinación permitió construir una pantalla organizada y similar al diseño planteado en los wireframes.

6. Productos incluidos

La pantalla incluye productos realistas acompañados por imágenes:

- Pimiento rojo
- Huevos
- Bananas
- Jengibre

Cada producto tiene una imagen, nombre, unidad de venta, cantidad y precio. Las imágenes fueron agregadas dentro de la carpeta drawable del proyecto para enriquecer visualmente la pantalla y acercarla al diseño propuesto.

7. Interacciones implementadas

La pantalla cuenta con varias interacciones realizadas desde Java.

Aumentar cantidad

Cada producto tiene un botón +. Al presionarlo, la cantidad del producto aumenta en uno y el precio del producto se actualiza según la nueva cantidad.

Por ejemplo, si el pimiento cuesta \$749.99 y el usuario aumenta la cantidad a 2, el precio mostrado para ese producto se actualiza multiplicando el precio unitario por la cantidad.

Disminuir cantidad

Cada producto tiene un botón -. Al presionarlo, la cantidad disminuye en uno, siempre que la cantidad actual sea mayor a 1.

Se evita que la cantidad llegue a 0 o a valores negativos, ya que eso no tendría sentido dentro del carrito.

Eliminar producto

Cada producto tiene un botón X. Al presionarlo, el producto desaparece del carrito junto con su línea divisoria.

Para esto se utiliza `View.GONE`, de manera que el elemento no solo se oculta visualmente, sino que también deja de ocupar espacio dentro del layout.

Ir al pago

Al presionar el botón **Ir al pago**, la aplicación calcula el total estimado del carrito y muestra un resumen del pedido.

Este resumen se crea dinámicamente desde Java mediante un `TextView`, y luego se agrega al contenedor del carrito usando `addView()`.

Esto permite cumplir con el requisito de agregar al menos un elemento dinámicamente desde Java, fuera del XML.

Modificación del carrito luego de calcular el total

Si el usuario ya generó el resumen del pedido y luego modifica cantidades o elimina productos, el resumen se oculta.

Esto se decidió porque el total mostrado corresponde al estado del carrito en el momento en que se presionó **Ir al pago**. Si el carrito cambia después, ese total deja de representar la compra actual.

Por eso, al modificar el carrito, se oculta el resumen y el usuario puede volver a presionar **Ir al pago** para calcular un nuevo total actualizado.

Carrito vacío

Si el usuario elimina todos los productos del carrito, se muestra un mensaje indicando que el carrito está vacío:

“Tu carrito está vacío. Agregá productos para continuar con la compra.”

Además, el botón **Ir al pago** se deshabilita y se muestra con menor opacidad, indicando visualmente que no se puede continuar con la compra si no hay productos.

8. Uso de ScrollView

La lista de productos está contenida dentro de un `ScrollView`.

Esto es importante porque, si el carrito contiene muchos productos o si el resumen del pedido queda ubicado debajo de la lista, el usuario puede desplazarse para ver todo el contenido.

De esta manera, la pantalla se adapta mejor a diferentes cantidades de productos y evita que información importante quede inaccesible.

9. Organización de recursos

Para mantener el proyecto más ordenado, se separaron distintos recursos en carpetas y archivos específicos.

Strings

Los textos principales se organizaron en `strings.xml`, incluyendo nombres de productos, textos de navegación, mensajes y textos de botones.

Esto evita escribir textos directamente en los layouts y permite mantenerlos centralizados.

Colores

Los colores principales del diseño se organizaron en `colors.xml`.

Entre ellos se incluyeron colores para:

- Verde principal.
- Texto principal.
- Texto secundario.
- Bordes.
- Separadores.
- Íconos de eliminar.

Imágenes

Las imágenes de productos se agregaron en la carpeta `drawable`.

También se agregaron íconos vectoriales para la barra de navegación inferior, como carrito, tienda, búsqueda, favoritos y usuario.

10. Barra de navegación inferior

Se implementó una barra inferior con cinco secciones:

- Tienda
- Explorar
- Carrito
- Favoritos
- Cuenta

La sección **Carrito** aparece destacada en color verde, indicando que es la pantalla activa.

Los íconos fueron creados como vector assets dentro de Android Studio y utilizados desde XML mediante `ImageView`.

11. Uso de Java

La lógica de interacción fue desarrollada en MainActivity.java.

Desde Java se manejan:

- Eventos de botones +.
- Eventos de botones -.
- Eventos de eliminación de productos.
- Cálculo del total estimado.
- Creación dinámica del resumen del pedido.
- Estado de carrito vacío.
- Deshabilitación del botón de pago cuando no hay productos.

La conexión entre XML y Java se realiza mediante findViewById().

12. Elemento dinámico creado desde Java

Para cumplir con el requisito de crear un elemento dinámicamente, se implementó un resumen del pedido que no existe originalmente en el XML.

El resumen se crea desde Java con:

```
TextView checkoutSummary = new TextView(this);
```

Luego, al presionar el botón **Ir al pago**, se agrega al contenedor del carrito mediante:

```
cartItemsContainer.addView(checkoutSummary);
```

Este elemento muestra el total estimado del carrito y se actualiza cada vez que el usuario vuelve a presionar el botón luego de modificar la compra.

13. Uso de Git y convenciones de commits

El desarrollo fue realizado directamente sobre un repositorio de GitHub.

Se realizaron varios commits durante el proceso, evitando entregar el proyecto en uno o dos commits grandes. Esto permite mostrar la evolución del trabajo paso a paso.

Se utilizó una convención de commits basada en prefijos como:

- feat: para nuevas funcionalidades.
- style: para cambios visuales o de estilo.
- refactor: para reorganización de código.
- docs: para documentación o informe.
- fix: para correcciones.

14. Conclusión

La pantalla **Mi carrito** fue seleccionada porque permite representar una parte importante del flujo de compra y, al mismo tiempo, incorporar varias interacciones útiles dentro de una sola pantalla.

La implementación cumple con los requisitos principales del trabajo: uso de layouts, textos, botones, imágenes, eventos, contenido realista, elementos dinámicos creados desde Java y trabajo versionado en GitHub.

Además, se agregaron comportamientos dinámicos como modificación de cantidades, eliminación de productos, cálculo de total, resumen del pedido y estado de carrito vacío, logrando una pantalla más completa y cercana a una aplicación real.